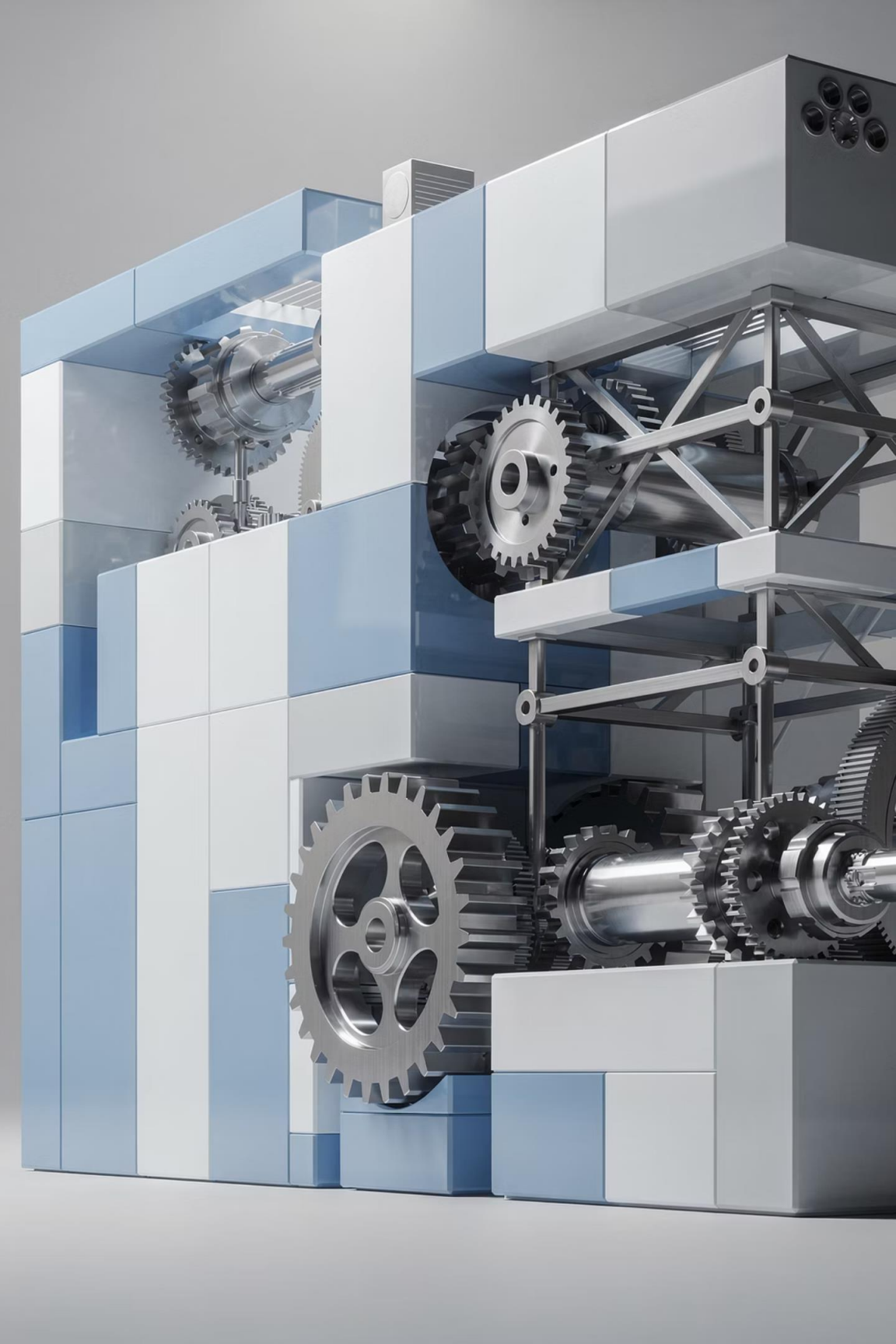




# Abstract Factory

Patrón de diseño creacional para producir familias de objetos relacionados sin especificar sus clases concretas.

# Contenido de la presentación

01

## Propósito

Qué es y para qué sirve el patrón

03

## Solución

Cómo resuelve el problema

05

## Pseudocódigo

Ejemplo práctico multiplataforma

02

## Problema

El escenario que motiva su uso

04

## Estructura

Componentes del patrón

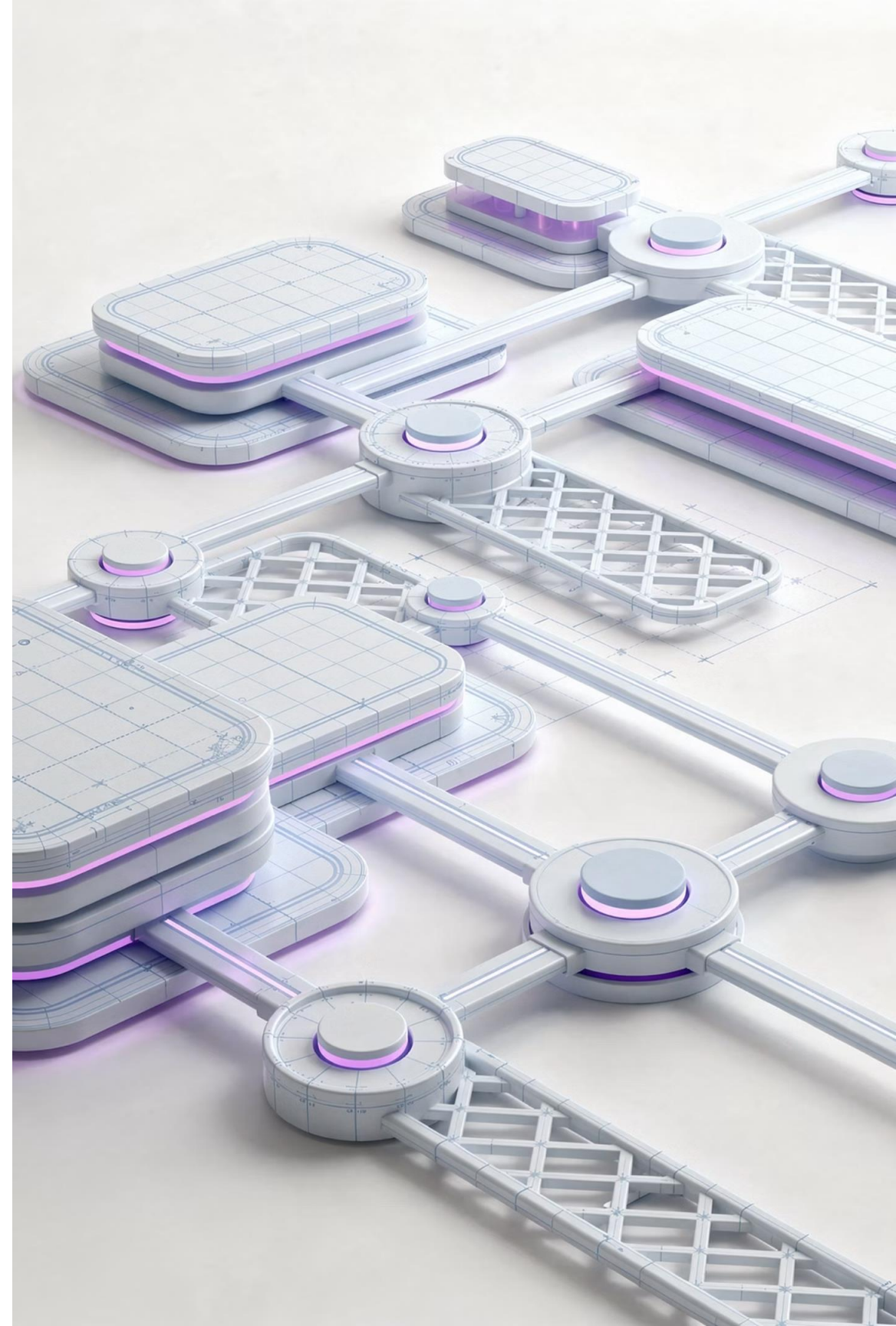
06

## Implementación

Pasos, pros, contras y relaciones

CAPÍTULO 1

# Propósito







# ¿Qué es Abstract Factory?

Abstract Factory es un patrón de diseño creacional que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.

En lugar de crear objetos directamente, delegamos la creación a una fábrica abstracta que garantiza que todos los productos generados sean compatibles entre sí. Esto desacopla el código cliente de las implementaciones específicas.





CAPÍTULO 2

# El Problema

# El simulador de tienda de muebles

Imagina que estás creando un simulador de tienda de muebles. Tu código está compuesto por clases que representan:

## Familia de productos

Productos relacionados que forman un conjunto:

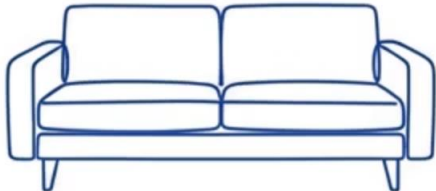
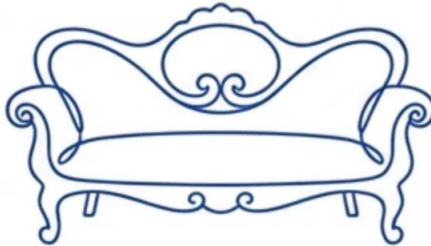
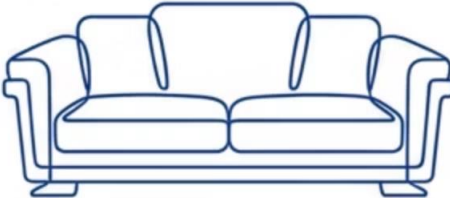
- Silla
- Sofá
- Mesilla

## Variantes disponibles

Cada producto existe en distintos estilos:

- Moderna
- Victoriana
- ArtDecó

# Familias de productos y sus variantes

|         | Moderna                                                                              | Victoriana                                                                            | ArtDecó                                                                               |
|---------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Silla   |    |    |    |
| Sofá    |   |   |   |
| Mesilla |  |  |  |



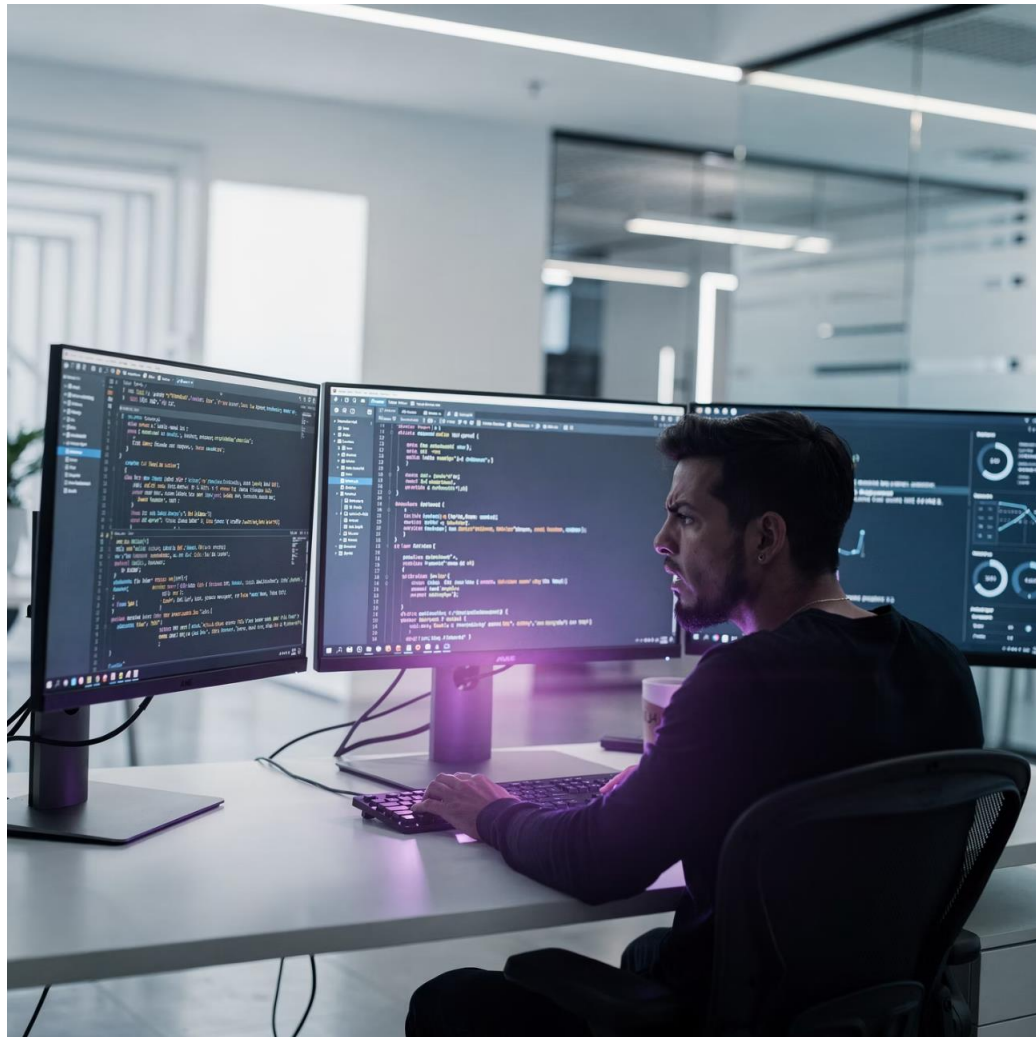
# El problema de la compatibilidad

Necesitamos crear objetos individuales de mobiliario que **combinen con otros objetos de la misma familia**. Los clientes se enfadan bastante cuando reciben muebles que no combinan.

- ❏ Un sofá de estilo moderno no combina con unas sillas de estilo victoriano. La coherencia estilística es fundamental.



# El reto de la extensibilidad



## No queremos modificar código existente

Los comerciantes de muebles actualizan sus catálogos muy a menudo. Debemos evitar tener que cambiar el código principal cada vez que se añadan **nuevos productos** o **familias de productos** al programa.

Necesitamos una arquitectura que permita crecer sin romper lo que ya funciona.



CAPÍTULO 3

# La Solución

# Paso 1: Interfaces para cada producto

Lo primero que sugiere el patrón es declarar de forma explícita **interfaces para cada producto diferente** de la familia (silla, sofá, mesilla). Todas las variantes de los productos siguen esas interfaces.

## Interfaz Silla

SillaModerna, SillaVictoriana y  
SillaArtDecó implementan esta interfaz

## Interfaz Sofá

SofáModerno, SofáVictoriano y  
SofáArtDecó implementan esta interfaz

## Interfaz Mesilla

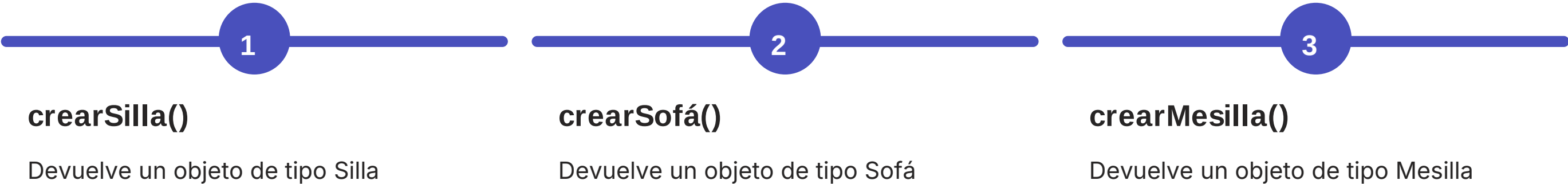
MesillaModerna, MesillaVictoriana y  
MesillaArtDecó implementan esta  
interfaz

Todas las variantes del mismo objeto se mueven a una **única jerarquía de clase**.



## Paso 2: La Fábrica Abstracta

Declaramos la **Fábrica Abstracta**: una interfaz con métodos de creación para todos los productos de la familia. Estos métodos devuelven productos abstractos representados por las interfaces extraídas previamente.



1

**crearSilla()**

Devuelve un objeto de tipo Silla

2

**crearSofá()**

Devuelve un objeto de tipo Sofá

3

**crearMesilla()**

Devuelve un objeto de tipo Mesilla

# Paso 3: Fábricas Concretas

Para cada variante de la familia de productos, creamos una **clase de fábrica independiente** basada en la interfaz `FábricaAbstracta`. Cada fábrica devuelve productos de un tipo particular.

1

## **FábricaModerna**

Crea `SillaModerna`, `SofáModerno`, `MesillaModerna`

2

## **FábricaVictoriana**

Crea `SillaVictoriana`, `SofáVictoriano`, `MesillaVictoriana`

3

## **FábricaArtDecó**

Crea `SillaArtDecó`, `SofáArtDecó`, `MesillaArtDecó`





# El código cliente trabaja con abstracciones

El código cliente funciona con fábricas y productos a través de sus **interfaces abstractas**. Esto permite cambiar el tipo de fábrica y la variante del producto sin descomponer el código cliente.

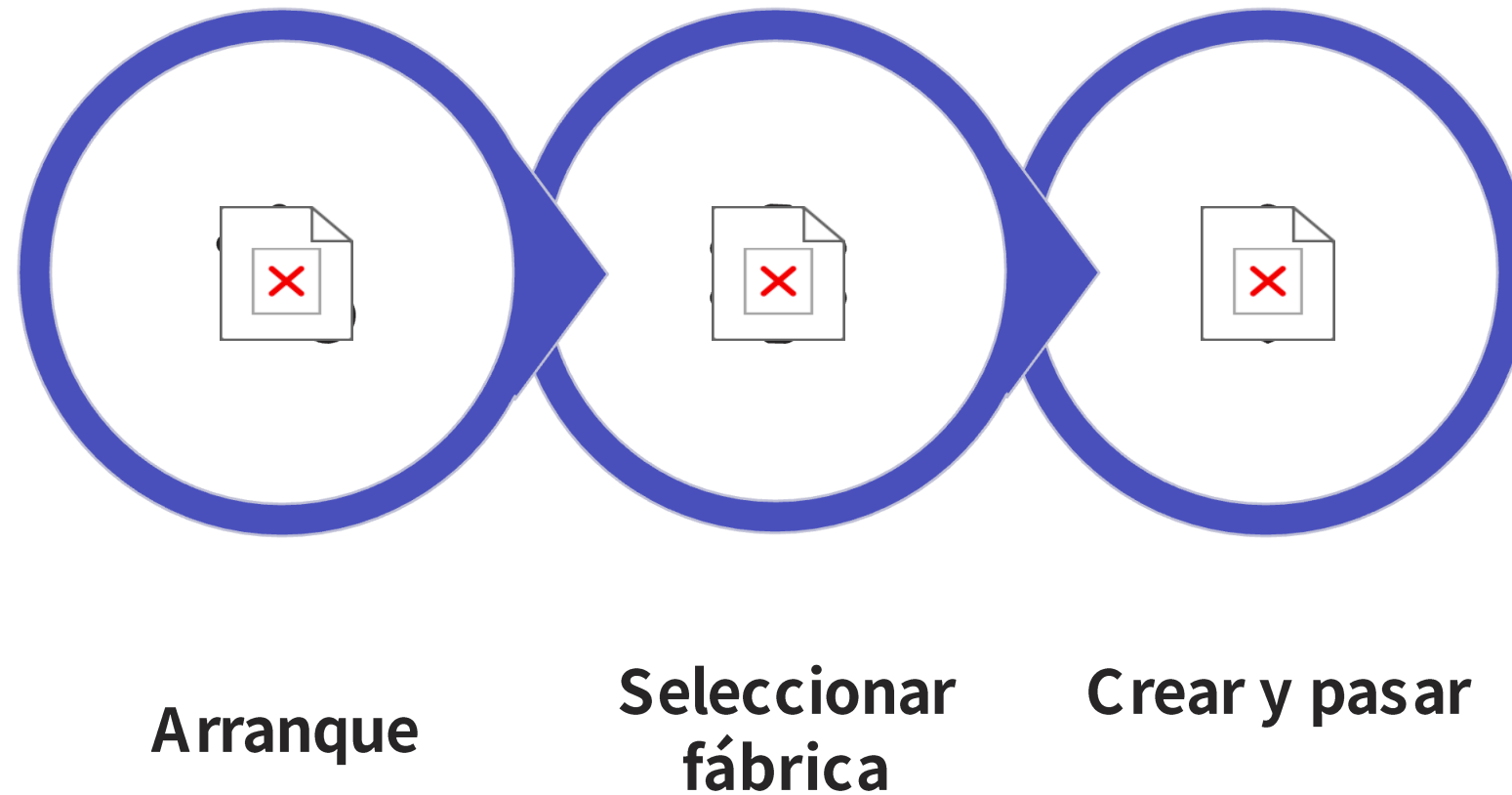
El cliente no tiene que conocer la clase de la fábrica ni el tipo concreto de silla que obtiene. Ya sea moderna o victoriana, trata a todas las sillas del mismo modo usando la interfaz abstracta **Silla**.

- Sea cual sea la variante de silla devuelta, siempre combinará con el sofá o mesilla producida por el mismo objeto de fábrica.





# ¿Cómo se crean los objetos de fábrica?

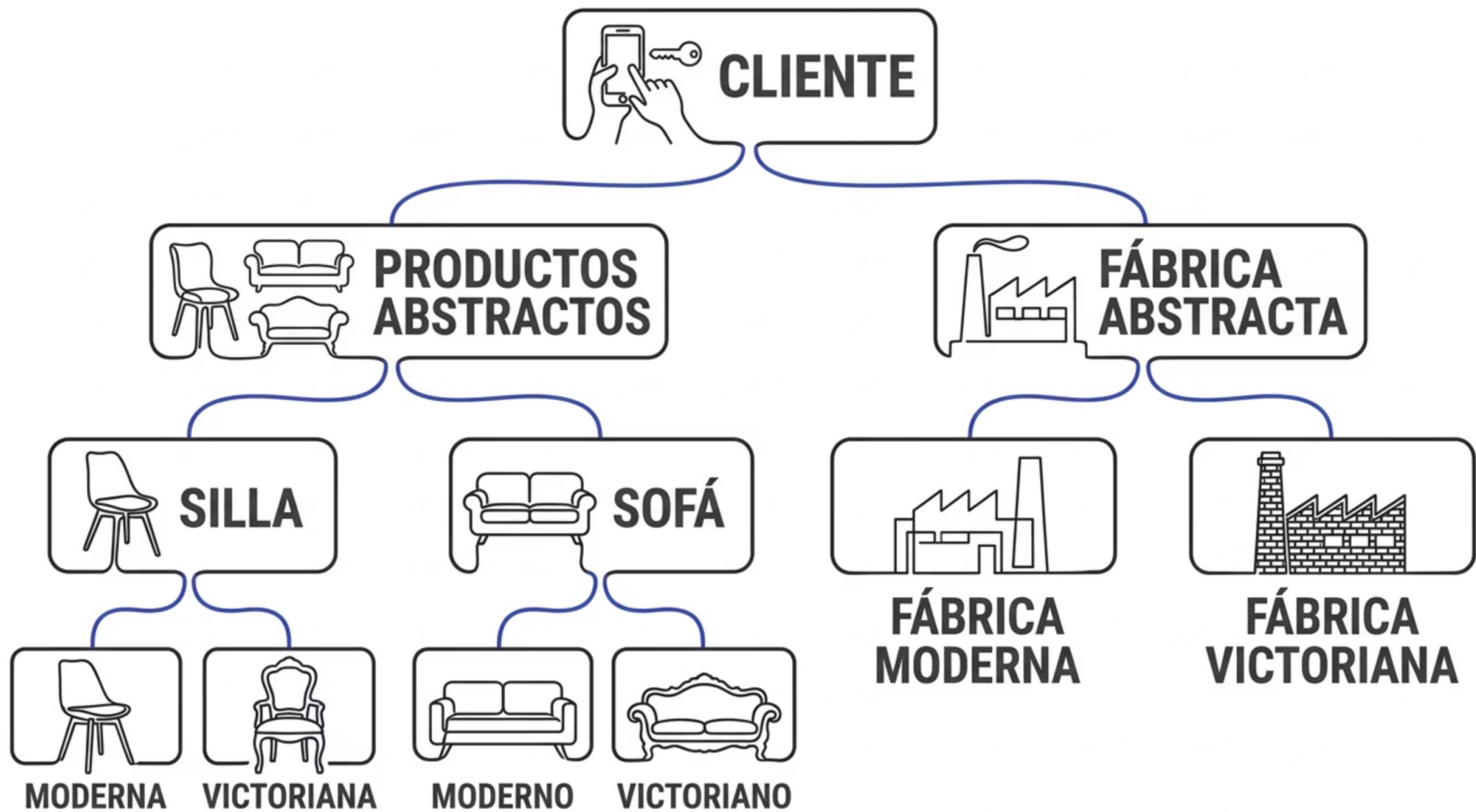


Normalmente, la aplicación crea un objeto de fábrica concreto en la **etapa de inicialización**, seleccionando el tipo de fábrica dependiendo de la configuración o los ajustes del entorno.



## CAPÍTULO 4

# Componentes del patrón





# Detalle de cada componente

## 1 Productos Abstractos

Declaran interfaces para un grupo de productos diferentes pero relacionados que forman una familia.

## 2 Productos Concretos

Implementaciones distintas de productos abstractos agrupados por variantes. Cada producto abstracto (silla/sofá) debe implementarse en todas las variantes (victoriano/moderno).

## 3 Fábrica Abstracta

Interfaz que declara un grupo de métodos para crear cada uno de los productos abstractos.

## 4 Fábricas Concretas

Implementan los métodos de creación. Cada fábrica concreta se corresponde con una variante específica y crea solo dichas variantes.

## 5 Cliente

Funciona con cualquier variante fábrica/producto concreta, comunicándose a través de interfaces abstractas. No se acopla a variantes específicas.

CAPÍTULO 5

# Pseudocódigo



# Ejemplo: UI multiplataforma

Este ejemplo ilustra cómo utilizar Abstract Factory para crear **elementos de interfaz de usuario (UI) multiplataforma** sin acoplar el código cliente a clases UI concretas, manteniendo la consistencia respecto al sistema operativo seleccionado.

## Windows

Botones y casillas con estilo Windows nativo

## macOS

Botones y casillas con estilo macOS nativo

Los mismos elementos UI deben comportarse de forma parecida, aunque tengan un aspecto diferente en distintos sistemas operativos. No queremos representar controles de macOS al ejecutarse en Windows.



# Flujo de ejecución



Cuando se lanza la aplicación, comprueba el tipo de sistema operativo actual y crea un objeto de fábrica de la clase correspondiente. El resto del código utiliza esta fábrica para crear elementos UI, evitando que se creen elementos equivocados.

# Interfaz GUIFactory y fábricas concretas

## Interfaz GUIFactory

```
interface GUIFactory is
    method createButton():Button
    method
createCheckbox():Checkbox
```

Declara métodos que devuelven productos abstractos. Los productos de una familia pueden colaborar entre sí.

## WinFactory

```
class WinFactory implements
GUIFactory is
    method createButton():Button is
        return new WinButton()
    method
createCheckbox():Checkbox is
        return new WinCheckbox()
```

## MacFactory

```
class MacFactory implements
GUIFactory is
    method createButton():Button is
        return new MacButton()
    method createCheckbox():Checkbox
is
        return new MacCheckbox()
```

# Productos abstractos y concretos

## Interfaz Button

```
interface Button is
    method paint()

class WinButton implements Button is
    method paint() is
        // Botón estilo Windows

class MacButton implements Button is
    method paint() is
        // Botón estilo macOS
```

## Interfaz Checkbox

```
interface Checkbox is
    method paint()

class WinCheckbox implements Checkbox is
    method paint() is
        // Casilla estilo Windows

class MacCheckbox implements Checkbox is
    method paint() is
        // Casilla estilo macOS
```

Cada producto individual tiene una **interfaz base**. Solo entre productos de la misma variante concreta es posible una interacción adecuada.



# Código cliente: clase Application

```
class Application is
  private field factory: GUIFactory
  private field button: Button
  constructor Application(factory: GUIFactory) is
    this.factory = factory
  method createUI() is
    this.button = factory.createButton()
  method paint() is
    button.paint()
```

El código cliente funciona con fábricas y productos **únicamente a través de tipos abstractos**: GUIFactory, Button y Checkbox. Esto permite pasar cualquier subclase fábrica o producto sin descomponer el código.

# Inicialización: selección de fábrica

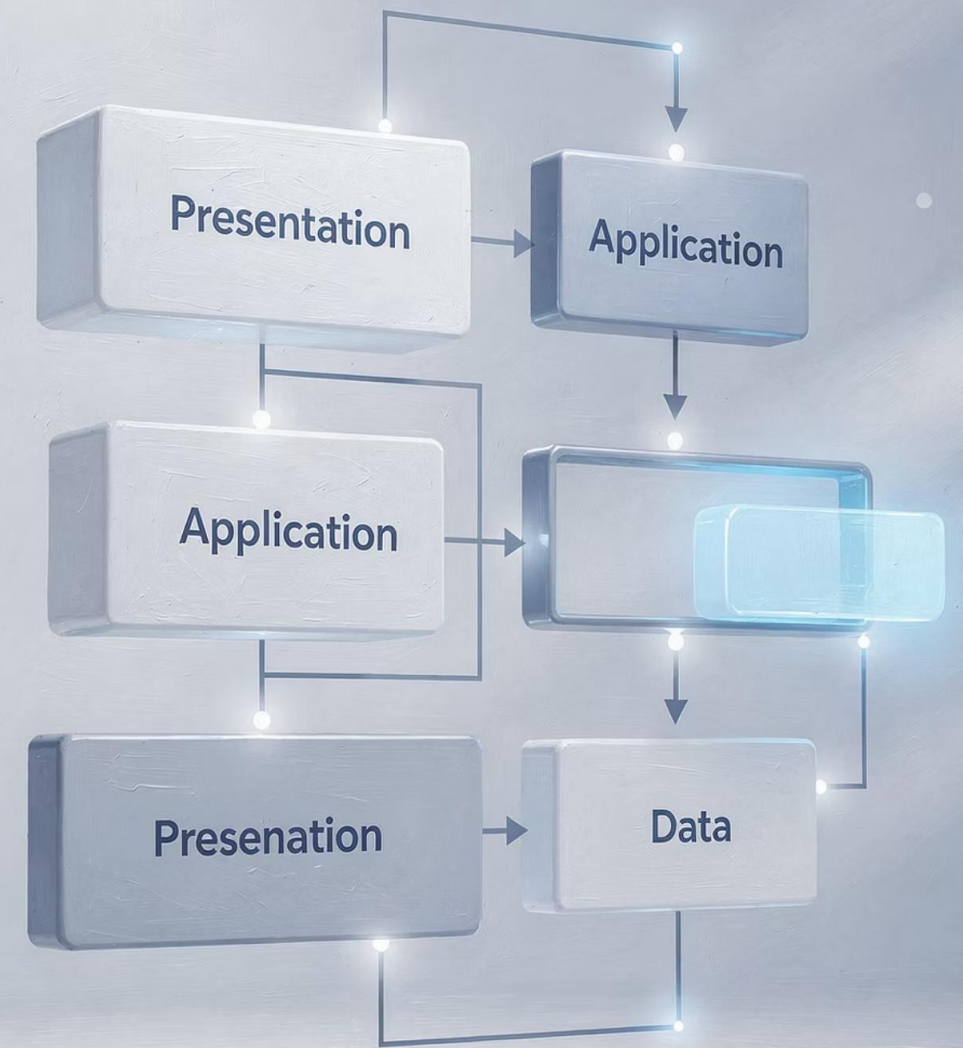
```
class ApplicationConfigurator is
  method main() is
    config = readApplicationConfigFile()
    if (config.OS == "Windows") then
      factory = new WinFactory()
    else if (config.OS == "Mac") then
      factory = new MacFactory()
    else
      throw new Exception("Error! Unknown OS.")
    Application app = new Application(factory)
```

- ❏ La aplicación elige el tipo de fábrica dependiendo de la configuración actual o los ajustes del entorno, creándola durante el tiempo de ejecución en la etapa de inicialización.

# Beneficio clave del ejemplo

Con este sistema, el código cliente **no depende de clases concretas** de fábricas y elementos UI, siempre que trabaje con objetos a través de sus interfaces abstractas. Esto permite soportar otras fábricas o elementos UI que añadamos más adelante.

No necesitas modificar el código cliente cada vez que añades una nueva variedad de elementos UI. Tan solo debes crear una **nueva clase de fábrica** y modificar ligeramente el código de inicialización.



# Aplicabilidad





# ¿Cuándo usar Abstract Factory?

## Familias de productos relacionados

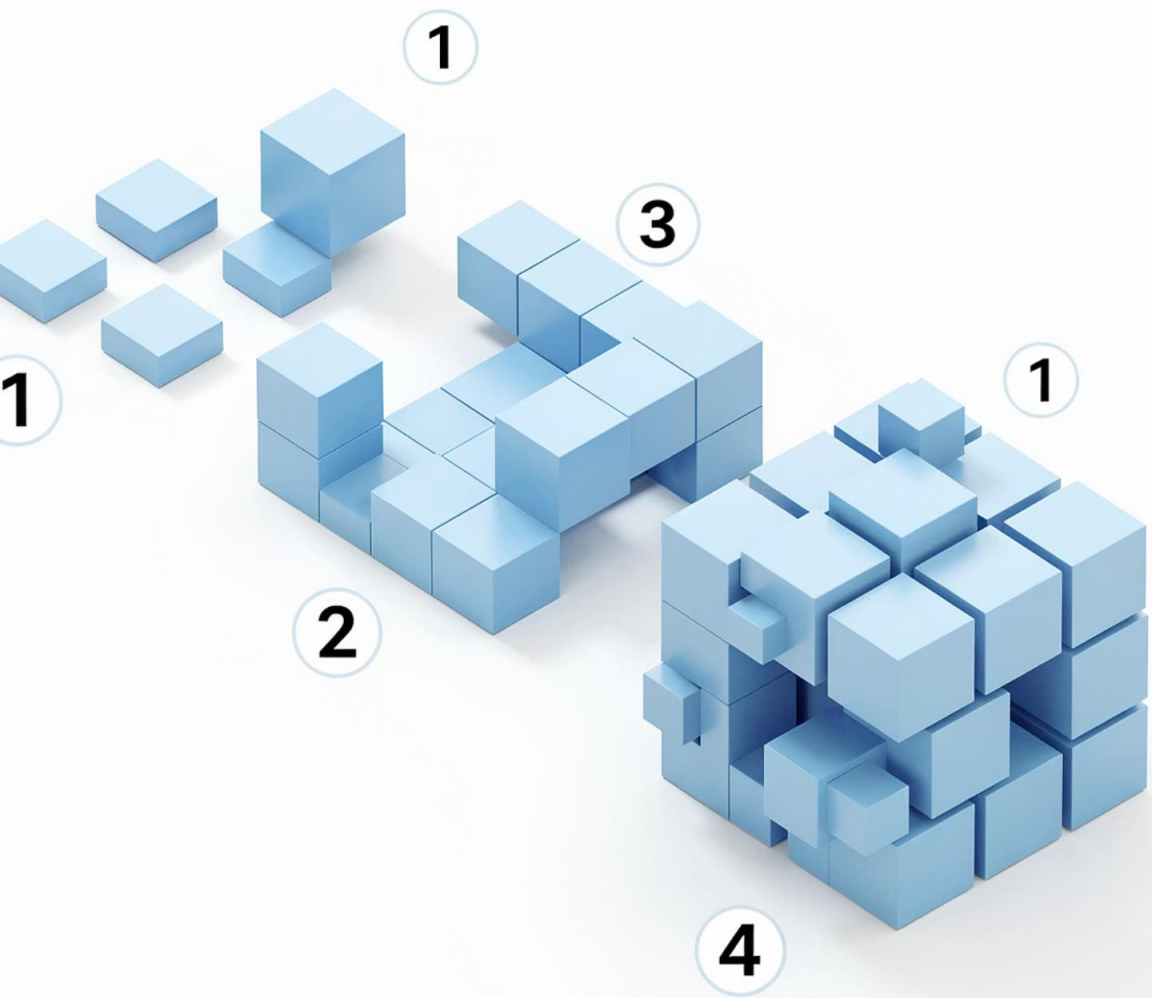
Cuando tu código deba funcionar con varias familias de productos relacionados, pero no desees que dependa de las clases concretas. Puede que no los conozcas de antemano o quieras permitir **futura extensibilidad**.

## Evitar variantes erróneas

Mientras tu código cree objetos a través de la interfaz de la fábrica, no tendrás que preocuparte por crear la **variante errónea** de un producto que no combine con los que ya ha creado tu aplicación.

## Responsabilidad única

Cuando tengas una clase con un grupo de métodos de fábrica que nublen su responsabilidad principal. Puede valer la pena extraer esos métodos en una **clase única de fábrica** o una implementación completa del patrón.



CAPÍTULO 7

# Implementación

# Cómo implementarlo paso a paso

## 1. Mapear la matriz

Mapea distintos tipos de productos frente a variantes de dichos productos.

## 2. Interfaces abstractas

Declara interfaces abstractas de producto para todos los tipos. Haz que todas las clases concretas las implementen.

## 3. Fábrica abstracta

Declara la interfaz de la fábrica abstracta con métodos de creación para todos los productos abstractos.

## 4. Fábricas concretas

Implementa una clase concreta de fábrica por cada variante de producto.

## 5. Inicialización

Crea código de inicialización que instancie la fábrica concreta según la configuración del entorno.

## 6. Refactorizar

Sustituye todas las llamadas directas a constructores de producto por llamadas al método de creación del objeto de fábrica.

CAPÍTULO 8

# Pros y Contras





# Ventajas del patrón



## Compatibilidad garantizada

Puedes tener la certeza de que los productos que obtienes de una fábrica son **compatibles entre sí**.



## Responsabilidad única

Puedes mover el código de creación de productos a un solo lugar, haciendo que el código sea más fácil de **mantener**.



## Bajo acoplamiento

Evitas un **acoplamiento fuerte** entre productos concretos y el código cliente.



## Principio abierto/cerrado

Puedes introducir nuevas variantes de productos **sin descomponer** el código cliente existente.

# Desventaja principal



## Complejidad añadida

Puede ser que el código se complique más de lo que debería, ya que se introducen **muchas nuevas interfaces y clases** junto al patrón.

Es importante evaluar si la complejidad adicional está justificada por los beneficios que aporta en tu caso de uso específico.

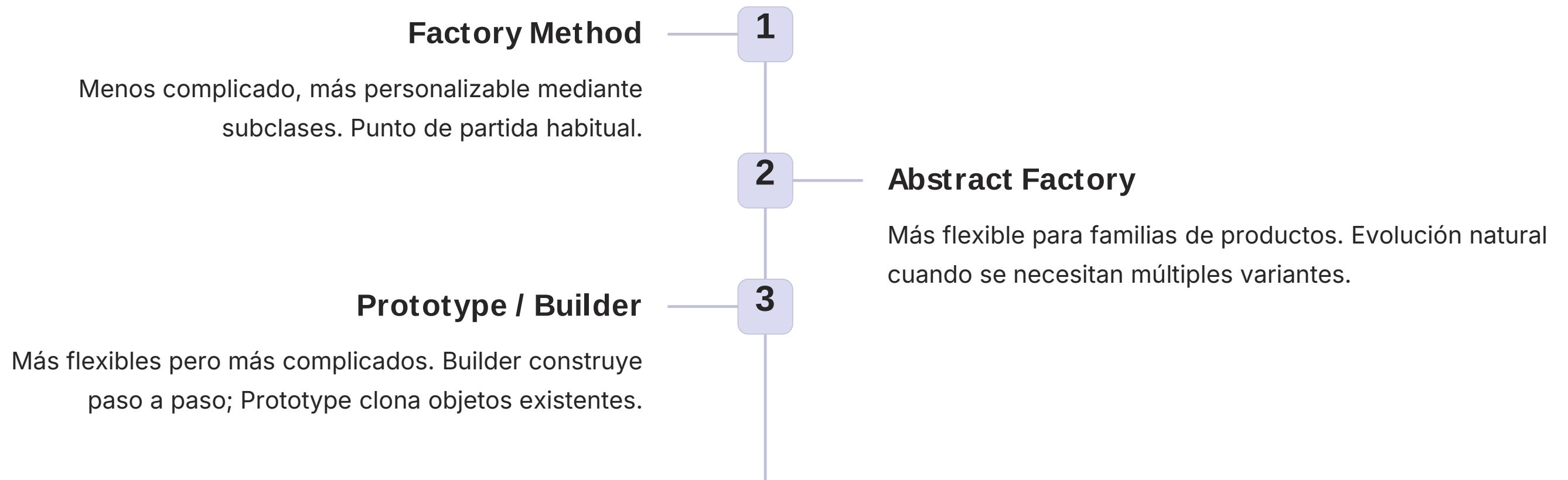
- ❏ Valora si realmente necesitas el patrón completo o si un Factory Method más simple sería suficiente para tu escenario.

CAPÍTULO 9

# Relaciones con otros patrones



# Evolución desde Factory Method



Muchos diseños empiezan con Factory Method y evolucionan hacia Abstract Factory, Prototype o Builder según crecen las necesidades.



# Abstract Factory vs Builder

## Abstract Factory

Se especializa en crear familias de objetos relacionados.  
Devuelve el producto inmediatamente.

## Builder

Se enfoca en construir **objetos complejos paso a paso**. Permite ejecutar pasos adicionales antes de extraer el producto.

# Combinaciones con otros patrones



## Con Factory Method

Las clases de Abstract Factory a menudo se basan en un grupo de métodos de fábrica, aunque también puedes usar **Prototype** para escribir los métodos.



## Como alternativa a Facade

Abstract Factory puede servir como alternativa a Facade cuando deseas **esconder la forma en que se crean los objetos** del subsistema.



## Con Bridge

Útil cuando algunas abstracciones definidas por Bridge solo pueden funcionar con **implementaciones específicas**. Abstract Factory encapsula estas relaciones.



## Como Singleton

Abstract Factory, Builder y Prototype pueden todos implementarse como **Singletons**.



# Resumen: Abstract Factory

## Propósito

Producir familias de objetos relacionados sin especificar clases concretas

## Clave

Interfaces abstractas para productos y fábricas desacoplan el código cliente

## Beneficio

Extensibilidad y compatibilidad garantizada entre productos